SOFTWARE ENGINEERING

Project Proposal

PrivateFind

An AI-Assisted Campus Lost and Found System with Private Ownership Verification and Smart Locker Handover

Field	Details
Course	Software Engineering
Instructor	Rajendra Dhakal, Ph.D.
University	Sejong University
Team name	Roaster
Team members	Zohidjon Mahmudjonov Murodjon Ugli; Umarov Muslimbek; Gerthofer Simon; Falk Raphael
Project category	AI-Assisted Web Application with Embedded and IoT Component
Submission date	18 September 2026
Revision	Week 4 update: work breakdown structure, COCOMO estimate and Gantt chart with staff allocation added (sections 5.2, 5.3 and 6.2 to 6.4)

Contents

Executive Summary **3**

1 Project Title and Team Roster **3**

1.1 Project Title 3

1.2 Team Roster 3

1.3 Team Coordination 3

2 System Architecture **3**

2.1 Mandatory Embedded and IoT Pipeline 4

2.2 Layer Responsibilities 4

2.3 Runtime Data Flow 4

3 Tools Used and Technology Stack **5**

3.1 Hardware Components 5

4 Description and Working Principle **6**

4.1 Problem Addressed 6

4.2 Main User Functions 6

4.3 AI-Assisted Matching Engine 6

4.4 Ownership Evidence and Verification 7

4.5 Smart Locker Handover 7

4.6 Boundary Conditions 7

5 Work Distribution **7**

5.1 Task Breakdown Matrix 8

5.2 Work Breakdown Structure 8

5.3 Effort Estimation with COCOMO 9

6 Detailed Timeline Weeks 1 to 13 ..... **11**

6.1 Milestones 11

6.2 Project Schedule (Gantt Chart) 12

6.3 Staff Allocation 12

6.4 Critical Path and Schedule Monitoring 12

7 Work Completed Before Week 3 ..... **13**

7.1 Team Formation and Skill Matrix 14

7.2 Initial Ideation and Theme Selection 14

7.3 Problem Statement Personas and Scope 14

7.4 Initial System Architecture Sketch 15

8 Development Process and Main Risks **15**

8.1 Selected Development Process 15

8.2 Main Risks 16

9 References **16**

Executive Summary

PrivateFind is an AI-assisted campus lost-and-found system designed around two practical constraints: returning a found item must take very little effort, and a likely match must not be treated as proof of ownership. A finder scans the QR code on the smart locker, uploads one to three photos, adds a short location note and deposits the item. The system keeps the full photo and identifying details private.

A multimodal AI service converts the finder photo and note into structured attributes. An owner first submits a lost-item report, and a hybrid Matching Engine ranks possible found items with a transparent score. The score only identifies candidates; it never proves ownership or opens the locker.

Ownership is checked separately: the claimant describes private details before seeing the full item information, and a deterministic Verification Engine applies the final rules, limits attempts and sends uncertain cases to administrator review. Only an approved claim receives a single-use pickup code. The project uses Python and FastAPI, an off-the-shelf AI API, a two-compartment locker and a browser-based interface.

1 Project Title and Team Roster

1.1 Project Title

PrivateFind An AI-Assisted Campus Lost and Found System with Private Ownership Verification and Smart Locker Handover

1.2 Team Roster

Name	Student ID	Primary role	Contact
Zohidjon Mahmudjonov Murodjon Ugli	23013058	Team Leader / AI and Backend Engineer	zohidjon.mah@gmail.com
Umarov Muslimbek	23012870	UI and Frontend Developer	muslimbekumarov8@gmail.com
Gerthofer Simon	26040296	Database Architect / Data Layer Developer	simon.gerthofer@gmail.com
Falk Raphael	26040280	IoT and Hardware Specialist / Locker Integration	1401raphael@gmail.com

Zohidjon is the team leader and owns AI and backend integration. Muslimbek owns UI and frontend, Simon owns the database and data layer, and Raphael owns IoT, hardware and locker integration. Each module therefore has one clearly accountable owner.

1.3 Team Coordination

The team uses a shared GitHub repository for source code, database scripts and project documentation. Zohidjon coordinates the weekly meeting, integration priorities and submission review. Short coordination takes place in the team messaging group, while every member remains responsible for completing and testing the deliverables assigned to their module.

2 System Architecture

PrivateFind uses a layered Python architecture. The browser serves the QR deposit flow, lost-item reporting, candidate results and claim status. A FastAPI application owns the workflow, REST endpoints and security rules. SQLAlchemy accesses MySQL, while the AI service is called only through the backend. The ESP32 also communicates only with the FastAPI REST API and never accesses the database or AI service directly.

User interfaces	FastAPI backend	Smart locker
Finder QR page Owner and admin web pages	Item workflow Matching and verification Deposit and pickup codes	ESP32 Keypad and LCD Presence sensors and servo locks
Photo and text input	MySQL through SQLAlchemy	Authenticated REST over Wi-Fi
	AI service Image and text attribute extraction Semantic similarity	No direct database or AI access

Figure 1 High level architecture and controlled data flow

2.1 Mandatory Embedded and IoT Pipeline

The embedded path follows the course requirement. A QR code on the locker opens the finder page, while the keypad and presence sensors connect to the ESP32. The ESP32 sends authenticated HTTP requests to FastAPI. The backend validates every deposit or pickup action, updates MySQL and returns only the command needed by the device.

Finder QR page plus keypad and presence sensor
ESP32 embedded device
Authenticated HTTP REST over Wi-Fi
FastAPI backend
SQLAlchemy and MySQL
Browser interface for owner and administrator

Figure 2 Embedded pipeline from locker input to backend database and web interface

2.2 Layer Responsibilities

Layer	Responsibility
Presentation	QR-based finder page, photo upload, lost-item reports, candidate results, claim status and administrator review.
Application	FastAPI services for authentication, item workflow, match scoring, verification policy, code lifecycle and locker REST endpoints.
AI support	Extract structured attributes from images and text, calculate semantic similarity and compare differently worded evidence. AI does not approve ownership or issue locker commands.
Data	SQLAlchemy models and MySQL tables for users, reports, photos, attributes, ownership evidence, claims, lockers, codes and audit records.
Hardware	ESP32, sensors, keypad, LCD and servo locks. The device opens a compartment only after backend approval.

2.3 Runtime Data Flow

1. The finder scans the locker QR code (no account needed), uploads one to three photos, selects the location and adds a short note. The AI service extracts structured attributes; the full photo and distinctive details stay private.
2. The backend assigns a free compartment and a short-lived deposit code. The finder enters the code, deposits the item and closes the door; the presence sensor confirms occupancy.

3. The owner signs in with a university account and submits a lost-item report (category, time, location, brand or model, unique marks) before viewing candidates.
4. The Matching Engine calculates a 0 to 100 score and returns only the top plausible candidates with limited public information.
5. The claimant submits private ownership evidence. The Verification Engine approves, rejects or sends the claim to administrator review; an approved claim reserves the item and receives a single-use pickup code.
6. The ESP32 validates the pickup code through FastAPI. After collection the code expires, the item is marked returned, the compartment is freed and the action is logged.

3 Tools Used and Technology Stack

The stack uses familiar web and database tools. AI is limited to hosted image and text analysis; the team will not train a model. This keeps the project focused on software design, integration, security and testing.

Area	Technology	Purpose
Programming	Python 3.12	Backend, scoring and integration code.
Backend	FastAPI and Pydantic	REST endpoints, request validation and application services.
Persistence	SQLAlchemy 2 and Alembic	Database mapping and schema migrations.
Database	MySQL 8	Reports, attributes, evidence, claims, locker state and audit data.
Authentication	Secure sessions or JWT plus Argon2 or BCrypt	University account login and role-based access.
Frontend	HTML5, Jinja2, Bootstrap 5 and JavaScript	Responsive finder, owner and administrator interfaces.
AI services	Multimodal model API and text embeddings	Photo attribute extraction and semantic similarity without custom training.
Image handling	Pillow and protected server storage	Basic validation, resizing and private photo storage for the prototype.
UI design	Figma	Wireframes and screen design.
Embedded	ESP32 and Arduino C++	Locker control and device-to-backend communication.
Testing	pytest and HTTPX	Unit, API and integration tests.
Version control	Git and GitHub	Source control, pull requests and issue tracking.
Dependencies	pip or uv with a locked requirements file	Repeatable project setup.

3.1 Hardware Components

Component	Quantity	Function
ESP32 development board	1	Runs the locker firmware and Wi-Fi client.
Infrared presence sensor	1 per compartment	Detects whether a compartment contains an item.
4x4 matrix keypad	1	Input for deposit and pickup codes.
SG90 servo motor	1 per compartment	Opens the compartment lock in the prototype.

Component	Quantity	Function
16x2 LCD display	1	Shows short instructions and success/error messages.
Power supply, wiring and enclosure	1 set	Physical prototype. Two compartments are sufficient for the demonstration.

4 Description and Working Principle

4.1 Problem Addressed

Campus lost-and-found systems face three related problems. A finder has little reason to spend several minutes registering and writing detailed questions. Publicly displaying full photos makes false claims easier. Finally, a good match is still not proof that the claimant owns the object. PrivateFind reduces finder effort, hides ownership evidence and separates candidate matching from the final verification decision.

The smart locker completes the same workflow. The finder can deposit the object quickly without sharing contact details, and a verified owner can collect it later without arranging a meeting.

4.2 Main User Functions

Function	Description
Quick found-item report	Finder scans the locker QR code, uploads photos and adds a short location note without creating an account.
AI attribute extraction	System converts the image and note into structured attributes and separates broad public data from private identifying evidence.
Deposit item	System assigns a free compartment and creates a short-lived deposit code.
Report lost item	Owner signs in and describes the item before seeing candidate details.
Rank candidates	Hybrid scoring combines structured matches and semantic similarity and returns the top plausible items.
Submit ownership evidence	Claimant describes private features, identifiers, damage, accessories or other details not shown publicly.
Verify claim	Deterministic rules apply the evidence score, critical mismatches and attempt limits.
Administrator review	Borderline claims or generic items with weak evidence require manual review.
Collect item	An approved claimant receives a single-use code validated by the backend before the compartment opens.
Audit actions	Reports, AI results, claim attempts, decisions, code use and locker events are recorded.

4.3 AI-Assisted Matching Engine

The Matching Engine first filters open found items by category and date range and then calculates a 0 to 100 Match Score from structured comparisons and semantic similarity. The AI service recognises equivalent wording, such as navy and dark blue, but the weighted score remains visible and testable.

Match Score = 25% category + 15% location + 15% time + 25% structured attributes + 20% semantic similarity. At most three candidates above a configurable threshold (initially 60) are shown. The score only marks a plausible match; it never proves ownership, reserves an item or creates a pickup code. Weights and threshold are tested with synthetic match and non-match cases.

4.4 Ownership Evidence and Verification

Ownership verification uses private evidence that the claimant submits before full item details are revealed, such as an exact model, a serial-number suffix, a hidden sticker, damage, contents or accessories. AI may compare differently worded descriptions, while deterministic rules calculate a weighted Ownership Score (strong evidence 3, distinctive 2, generic 1). A score of at least 80 without critical conflicts is approved; 50 to 79, or a generic item such as a plain umbrella, goes to administrator review; lower scores, a critical mismatch (for example a wrong serial suffix) or more than three failed attempts lead to rejection and a cooldown. The result never reveals which fact was wrong, and the first approved claim reserves the item in one database transaction.

Stage	AI support	Final system rule
Evidence capture	Extract attributes and distinctive visible features from finder photos.	Keep full photo and private attributes hidden from public listings.
Candidate matching	Calculate semantic similarity between lost and found descriptions.	Use the Match Score only to rank candidates.
Ownership comparison	Compare differently worded claimant evidence with stored private evidence.	Calculate weighted Ownership Score and check critical conflicts.
Decision	No authority to approve or unlock.	Approve, review or reject using fixed thresholds, attempt limits and reservation rules.

Figure 3 Separation of AI support from the final ownership decision

4.5 Smart Locker Handover

The locker uses two short-lived codes. A deposit code is created after the finder submits the QR form and opens one assigned empty compartment. A pickup code is issued only after ownership verification. Both codes are single-use, tied to one compartment and validated by FastAPI.

The ESP32 does not decide whether a claim or code is valid. It forwards the code and device identifier to the backend, waits for approval and then activates the correct servo. Presence sensors confirm the empty-to-occupied deposit transition and the occupied-to-empty pickup transition. The prototype uses opaque compartment doors so that stored items and their distinctive features are not visible to unauthorised users.

4.6 Boundary Conditions

- Account identity: owners and claimants use a university e-mail account. A finder may use the QR deposit session without registering, but the session is short-lived, tied to one locker and rate-limited.
- Expired code: an expired deposit or pickup code cannot open the locker. A valid replacement requires a new backend request and is recorded in the audit log.
- No free compartment: the QR page explains that deposit is unavailable; it does not accept an item that cannot be secured.
- AI or hardware unavailable: photo extraction and the locker API can be mocked during development. Failed AI jobs remain pending for retry or administrator correction; the final demonstration uses the physical ESP32 path.

5 Work Distribution

The work is divided across four accountable areas. Zohidjon leads the team and owns AI and backend integration; Muslimbek owns UI and frontend; Simon owns the database and matching data layer; and Raphael owns IoT, hardware and locker integration. The AI scope remains limited to API integration and score evaluation rather than model training. Section 5.2 breaks this work down into 54 work packages with 455 planned hours, and section 5.3 checks the total effort with COCOMO.

Member	Primary responsibility	Main deliverables
Zohidjon Mahmudjonov Murodjon Ugli	Team Leadership / AI / Backend	Team coordination, FastAPI setup, authentication, multimodal API integration, ownership verification policy, API tests and final system integration.
Umarov Muslimbek	UI / Frontend	Figma wireframes, Jinja2 and Bootstrap pages, QR finder flow, photo upload, lost report, candidate results and claim screens.
Gerthofer Simon	Database / Matching Data Layer	ER model, MySQL schema, SQLAlchemy models, Alembic migrations, hybrid Match Score, audit data and synthetic matching cases.
Falk Raphael	IoT / Hardware / Locker Integration	ESP32 firmware, sensors, keypad, LCD, servos, device authentication and deposit or pickup code flow.

5.1 Task Breakdown Matrix

Module or deliverable	Owner	Support
Software Requirements Specification	All	Zohidjon consolidates
Database model and migrations	Simon	Zohidjon reviews
Hybrid Matching Engine	Simon	Zohidjon integrates AI similarity
AI attribute extraction	Zohidjon	Muslimbek supports upload and correction UI
Audit log and attempt constraints	Simon	Zohidjon reviews
Wireframes and user flows	Muslimbek	All review
Finder owner and claim screens	Muslimbek	Zohidjon and Simon
Authentication and authorisation	Zohidjon	Simon supports user data
Ownership Verification Engine	Zohidjon	Simon supports transactions
Locker REST contract and code lifecycle	Raphael	Zohidjon
ESP32 firmware and physical locker	Raphael	Simon
Testing and final QA	Zohidjon	All members execute assigned tests

5.2 Work Breakdown Structure

The WBS divides PrivateFind into 12 phases (level 1) and 54 work packages (level 2), numbered as in the Gantt chart in section 6.2. Each package has one accountable owner, a checkable result and an effort estimate in hours per member. Most implementation packages take one to two weeks; continuous coordination (1.1) is tracked as recurring overhead, and only a few integration packages span three weeks. Figure 4 at the end of this chapter shows the complete structure.

Member	Primary role	Effort (h)	Share	Avg. h per week
Zohidjon Mahmudjonov	Team leader / AI and backend	134	29 %	10.3
Umarov Muslimbek	UI and frontend	100	22 %	7.7
Gerthofer Simon	Database / matching data layer	119	26 %	9.2
Falk Raphael	IoT / hardware / locker	102	22 %	7.8
Team		**455**	**100 %**	**35.0**

The team leader carries the largest share because of coordination, integration and the Verification Engine; Simon carries a moderately higher share because the database and Matching Engine span several phases. The hours include meetings, documentation, testing, deployment and the presentation, not only programming.

5.3 Effort Estimation with COCOMO

As a top-down check of the bottom-up WBS estimate, the size of each module was estimated with three values, optimistic (O), most likely (M) and pessimistic (P), and combined with the PERT formula (O + 4M + P) / 6. Only code written by the team is counted; tests and reused libraries (FastAPI, SQLAlchemy, Bootstrap, the hosted AI API, Arduino libraries) are excluded.

Module	Language	O	M	P	Expected
Authentication & user management	Python	250	400	600	408
Found-item reporting & QR deposit session	Python	300	450	700	467
AI service integration	Python	250	400	650	417
Hybrid Matching Engine	Python	250	400	600	408
Ownership Verification Engine	Python	300	450	700	467
Claims, reservation & admin review	Python	250	400	600	408
Locker REST API & code lifecycle	Python	200	350	550	358
Security: attempt/rate limits & audit log	Python	200	300	500	317
Data layer (SQLAlchemy models, Alembic migrations)	Python/SQL	400	600	900	617
Web UI (Jinja2 templates, Bootstrap, JavaScript)	HTML/JS	800	1,200	1,800	1,233
ESP32 locker firmware	Arduino C++	350	550	850	567
Total		**3,550**	**5,500**	**8,450**	**5,667**

The expected size is **5,667 LOC ≈ 5.67 KLOC**. Basic COCOMO (Boehm, 1981) is applied in **semi-detached mode** as a conservative assumption, because the project combines mixed team experience with interacting web, AI and embedded components; organic and embedded mode serve as lower and upper bounds (E = a × KLOC^b, D = 2.5 × E^d, Staff = E / D).

Mode	a / b	Effort (PM)	Duration (months)	Staff (persons)	Effort (h)*
Organic	2.4 / 1.05	14.8	7.0	2.1	2,254
Semi-detached (selected)	**3.0 / 1.12**	**20.9**	**7.2**	**2.9**	**3,182**
Embedded	3.6 / 1.2	28.9	7.3	3.9	4,387

* 1 person-month = 152 working hours. An Intermediate COCOMO cross-check (15 cost drivers, EAF 1.46, mainly due to the compressed schedule, SCED) gives 30.5 person-months. Basic COCOMO therefore expects about **3,182 hours over 7.2 months**, while the WBS plans **420 hours** for the directly comparable core development in Weeks 3 to 13 and **455 hours** overall, including post-development work in Weeks 14 and 15. The difference is treated as a scope-risk indicator rather than a staffing requirement, because PrivateFind is a small semester prototype built with modern frameworks, reused libraries and a hosted AI service. The team therefore keeps strict scope boundaries, uses mocks and fallback options, and re-estimates effort after the MVP in Week 8. The complete calculation, including all cost drivers, is kept in the planning workbook.

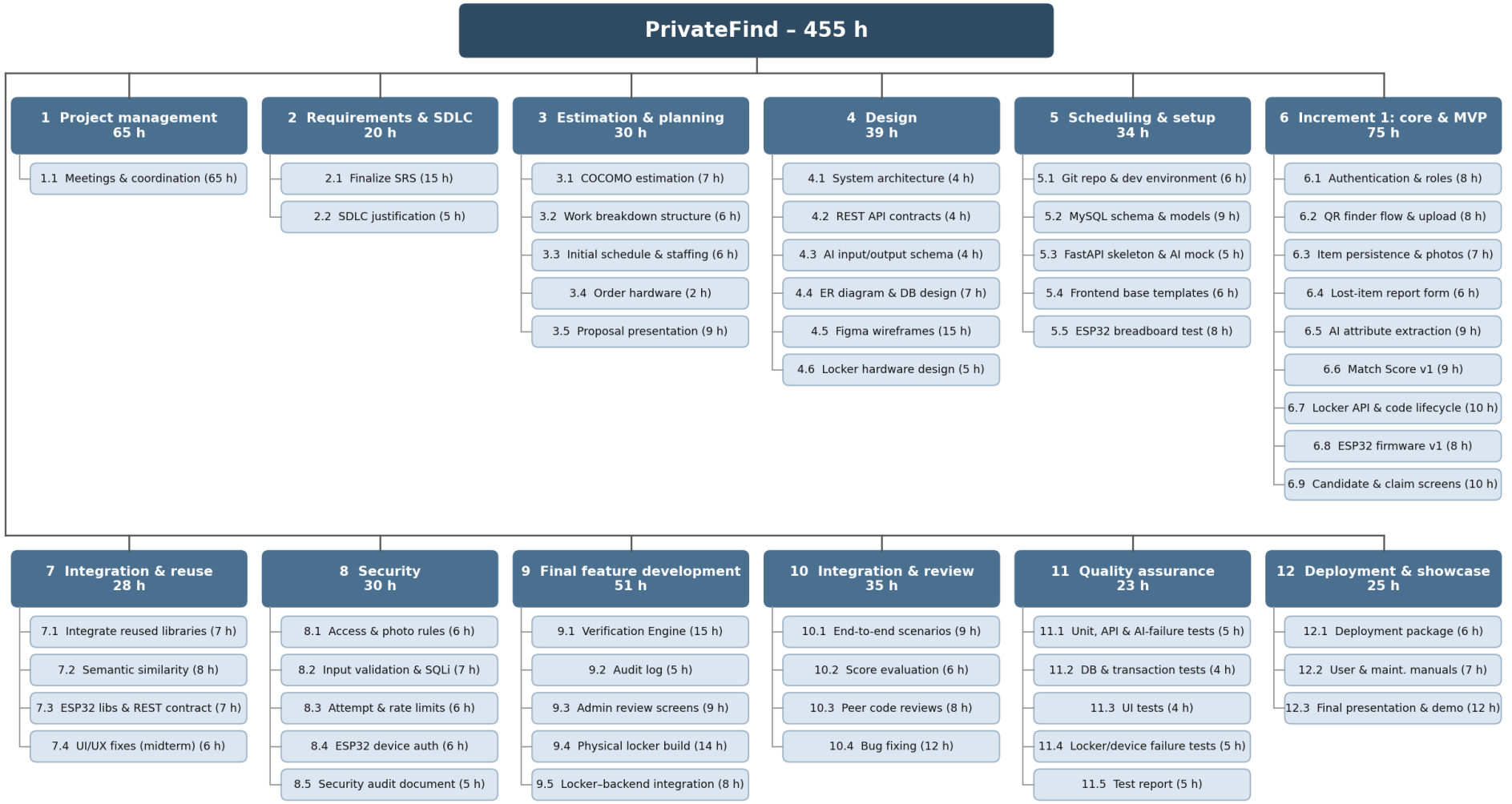


Figure 4 Work breakdown structure of PrivateFind with planned effort in hours (level 1: phases, level 2: work packages)

6 Detailed Timeline Weeks 1 to 13

The timeline follows the course project schedule; Weeks 14 and 15 are added for deployment and the final showcase.

Week	Course and project phase	PrivateFind work and deliverable
1	Ideation and Team Formation	Form team, select PrivateFind, define the core problem and draft the proposal.
2	Requirements Analysis	Define QR finder flow, owner flow, AI boundaries, verification rules, database, UI and IoT scope. Deliverable: SRS draft.
3	SDLC Selection	Finalize the SRS and justify incremental development. Deliverable: finalized SRS and SDLC justification.
4	Estimation and Work Breakdown	Estimate effort with COCOMO, build the WBS, create the initial Gantt chart with staff allocation and order the locker hardware. Deliverable: COCOMO estimation, WBS and initial schedule.
5	Architecture UI and Database Design	Finalize architecture, API contracts, AI input and output schema, wireframes and ER diagram; present the project proposal. Deliverable: system design specification.
6	Scheduling and Setup	Confirm the schedule baseline; set up repository, development environment, database schema, FastAPI skeleton, frontend templates and AI mock. Deliverable: schedule, repository URL and database baseline.
7	Core Logic Implementation	Implement QR reporting, photo upload, lost reports, authentication and database persistence. Deliverable: working core data flow.
8	MVP Checkpoint	Demonstrate finder report, AI attribute extraction, owner report, candidate score and claim submission using a mock locker.
9	Integration and Reuse	Integrate Bootstrap, FastAPI libraries, AI API and ESP32 peripheral libraries; finalize the locker REST contract.
10	Security Implementation	Complete access control, private-photo rules, server-side validation, attempt limits, rate limits and code expiry.
11	Sprint and Refinement (Final Feature Increment)	Complete matching, ownership verification, administrator review and physical locker integration.
12	Integration and Review	Run end-to-end scenarios, evaluate score cases, peer review code and fix defects.
13	Quality Assurance and Completion	Execute unit, API, database, AI-failure and locker tests. Deliverable: final source, executable and test report.
14	Deployment and Handover (post-development)	Prepare the deployment package and setup scripts, write the user and maintenance manuals.
15	Final Capstone Showcase (post-development)	Live demonstration and presentation of architecture, UI design, database model and SE methodology.

6.1 Milestones

Milestone	Week	Definition of done
M1 Requirements and SDLC approved	3	Team agrees on the finder effort, AI boundaries, verification rules and system scope.
M2 Estimation, WBS and initial schedule agreed	4	COCOMO estimate, WBS and initial Gantt chart with staff allocation are agreed by the team.

Milestone	Week	Definition of done
M3 Design approved	5	Architecture, ER model, AI data contract and wireframes are reviewed.
M4 Schedule, repository and database baseline ready	6	Schedule baseline is confirmed; repository, development environment and database schema are available.
M5 First running increment	7	QR reporting, owner reporting and database persistence work through FastAPI.
M6 MVP	8	An image becomes structured attributes and at least one candidate and claim can be produced end to end.
M7 Feature complete	11	Matching, verification, review and physical locker flow work together.
M8 Release candidate	12	Must-have features are integrated, evaluated and peer reviewed.
M9 Project complete	13	Testing is complete and no high-severity defect remains open.
M10 Final showcase	15	Live demonstration and final presentation are delivered.

6.2 Project Schedule (Gantt Chart)

The Gantt chart (Figure 5, end of this chapter) starts in Week 3; Weeks 1 and 2 are documented in section 7. Each bar is a WBS package coloured by its owner, with supporters named next to it; red diamonds mark milestones. It is maintained in the shared planning workbook.

6.3 Staff Allocation

Planned hours per member and week; each package's hours are spread evenly over its duration, including the weekly meeting, and rounded to whole hours (the workbook keeps exact values).

Member	W3	W4	W5	W6	W7	W8	W9	W10	W11	W12	W13	W14	W15	Total
Zohidjon	11	9	15	11	15	9	13	15	9	11	8	5	3	134
Muslimbek	4	8	9	8	14	10	7	9	9	8	6	6	2	100
Simon	6	13	12	10	14	5	9	13	8	13	8	6	2	119
Raphael	4	5	15	6	9	9	11	11	11	8	6	5	2	102
Team	**25**	**35**	**51**	**35**	**52**	**33**	**40**	**48**	**37**	**40**	**28**	**22**	**9**	**455**

The load averages about 9 hours per member and week, with peaks of up to 15 hours in Weeks 5, 7 and 10, so the plan is an upper-bound estimate; UI refinements or enclosure improvements are deferred if needed. As Raphael is the only hardware specialist, Simon supports the locker design (4.6) and build (9.4).

Critical non-staff resources are the ESP32 locker, the hosted AI API, the MySQL environment and the GitHub repository; the hardware is ordered in Week 4 for integration in Weeks 9 to 11.

6.4 Critical Path and Schedule Monitoring

The dependencies form a network: for every package the workbook calculates its float, i.e. how many weeks it can slip before a later package or a fixed course deadline is affected. Because the course sets a deliverable almost every week, the plan is deadline-driven and nearly all packages are critical; only 7.1 and 7.2 have one or two weeks of float (Figure 5). The longest chains are the software chain (4.4 → 5.2 → 6.3 → 6.6 → 6.9 → MVP → 9.1 → 9.3 → 10 → 11) and the hardware chain (3.4 → 5.5 → 6.8 → 7.3 → 9.4 → 9.5). A one-week delay of the Match Score (6.6), for example, would move the MVP to Week 9 and completion to Week 14. As the course deadlines cannot move, such a delay is absorbed by shifting support hours, using the fallbacks from section 5.3 or reducing scope. A delay entered in the workbook moves all dependent packages automatically and flags missed deadlines; the plan is reviewed in every weekly meeting and re-estimated after the MVP.

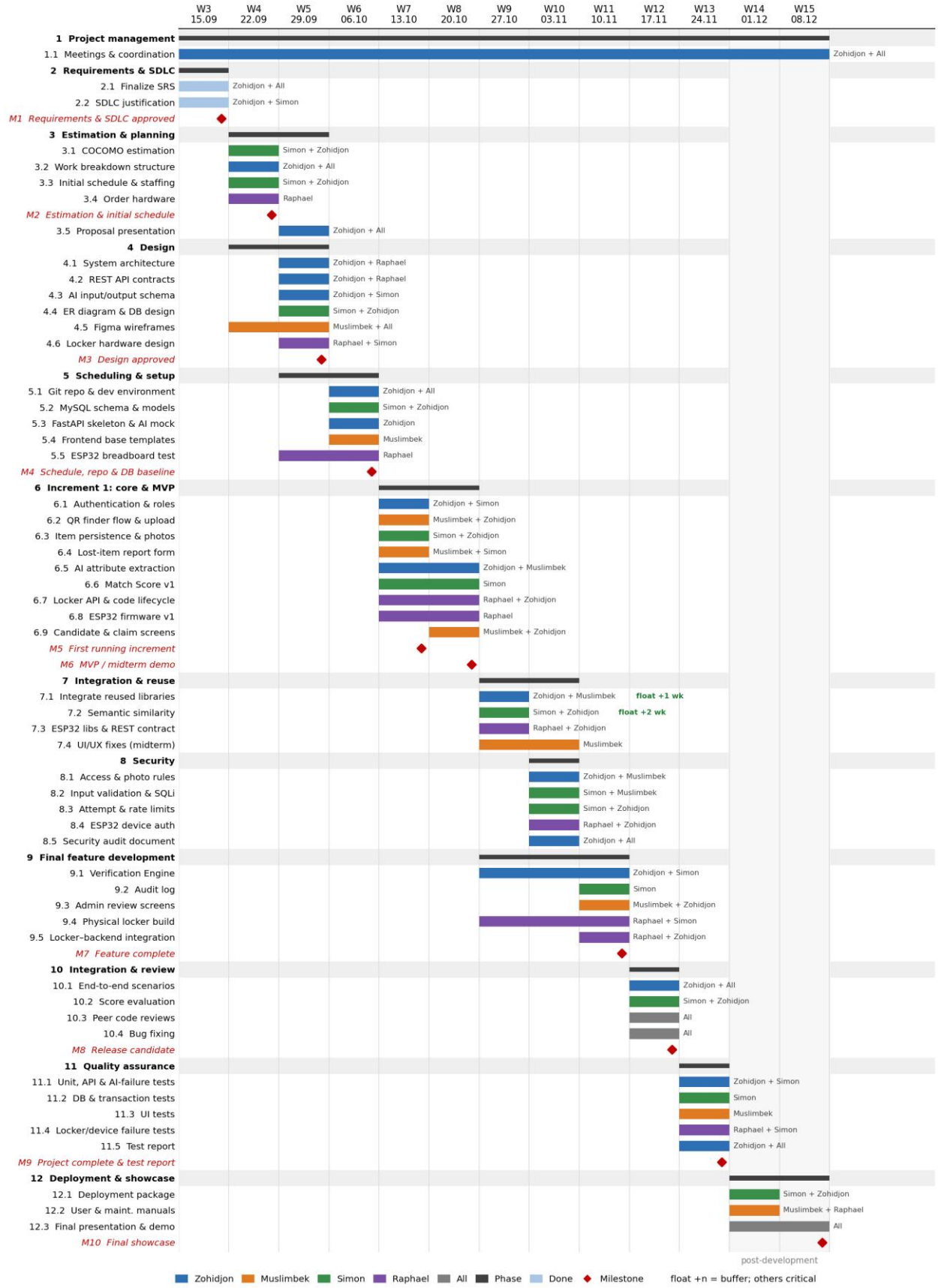


Figure 5 Gantt chart of PrivateFind, Weeks 3 to 15, with owner, support and float (packages without a float label are critical)

7 Work Completed Before Week 3

This section records the preliminary work completed up to the proposal submission on 18 September 2026. The last ideation session (17 September) already fell into Week 3 and is included for completeness; the Gantt chart therefore starts in Week 3.

7.1 Team Formation and Skill Matrix

Member	Frontend	Backend and AI	Database	Hardware
Zohidjon Mahmudjonov Murodjon Ugli	Working	Primary	Working	Basic
Umarov Muslimbek	Primary	Basic	Basic	Basic
Gerthofer Simon	Basic	Working	Primary	Support
Falk Raphael	Basic	Working	Support	Primary

Primary means accountable for the area; Working means able to implement or review it; Support means a secondary contributor; Basic means familiar enough to understand and test the work.

7.2 Initial Ideation and Theme Selection

Session	Main discussion	Decision
Session 1 8 September 2026	Compared a basic lost-and-found site, a stand-alone locker and a combined system. A CRUD-only site was too simple; hardware alone did not show enough software engineering.	Use ownership verification as the core problem and the locker for asynchronous handover.
Session 2 10 September 2026	Reviewed finder motivation. Registration, long descriptions and manually written questions create too much effort for a person who receives little direct benefit.	Use QR, photo and a short note. Do not require a finder account for the deposit session.
Session 3 17 September 2026	Reviewed AI matching and false-ownership risk. Full public photos could help dishonest claimants, while an AI match alone cannot prove ownership.	Use an off-the-shelf multimodal API for extraction, a transparent hybrid Match Score and a separate rule-based Ownership Score with administrator review.

Feasibility summary

The scope is feasible because it uses an existing multimodal API instead of model training, keeps the scoring logic explicit, uses a server-rendered frontend and limits the locker to two compartments; all modules can be tested with mocks before integration.

7.3 Problem Statement Personas and Scope

Problem statement. A finder needs a fast way to secure an item, while an owner needs useful matching without exposing the details that prove ownership. Public full photos and AI-only approval would increase false-claim risk. PrivateFind uses QR-based deposit, private image-derived evidence, candidate scoring and a separate verification policy before smart-locker release.

Persona	Situation and goal	Needs and constraints
Finder	Finds an item between classes and wants to hand it over immediately without registering or meeting a stranger.	QR access, photo plus short note, under one minute of work, clear locker instructions and no contact sharing.
Owner or claimant	Lost an item and wants likely candidates while proving ownership without public exposure of private details.	Useful ranking, clear evidence prompts, limited false claims, fair review and secure pickup.

Persona	Situation and goal	Needs and constraints
Administrator	Handles uncertain claims, AI errors and uncollected items.	Private evidence view, audit history and the ability to approve, reject or correct item data.

Preliminary functional requirements

Before Week 3 the team had defined the core requirements, which were later refined into the workflow in sections 4.2 to 4.6:

- Anonymous QR-based deposit with one to three photos, a location and a short note.
- AI attribute extraction with public attributes stored separately from private evidence.
- Owner accounts with university e-mail and a lost-item report before candidates are shown.
- Explainable 0 to 100 Match Score and a rule-based Ownership Score with approve, review or reject decisions.
- One-time deposit and pickup codes validated through FastAPI, attempt limits and an audit log.

Preliminary non-functional requirements

- Full photos and distinctive evidence are never shown in the public item list; the AI API key stays on the server.
- AI supports extraction and comparison but cannot approve ownership or open a locker.
- The web application and ESP32 use authenticated backend interfaces and never access MySQL directly.
- Normal web requests return within two seconds; AI processing completes within ten seconds in the demonstration.

Scope boundaries

In scope are the QR-based deposit, protected photo upload, AI attribute extraction, owner accounts, candidate scoring, private ownership evidence, rule-based verification, one-time locker codes, a two-compartment locker, audit logging and basic administrator review. Out of scope are model training, public photo browsing, automatic approval of generic items, rewards, a chatbot, a mobile app and integration with university systems.

Initial boundary conditions

These conditions were later refined into the operational rules in section 4.6. The two central ones were:

- Only one approved claim can reserve an item at a time; approval closes or blocks competing claims.
- Expired deposit and pickup codes cannot open the locker; the item remains secured until a valid backend action is completed.

7.4 Initial System Architecture Sketch

The revised architecture fixes four boundaries: the browser communicates with FastAPI; FastAPI alone accesses MySQL through SQLAlchemy; the backend alone calls the AI service; and the ESP32 communicates only through authenticated REST endpoints. The Matching Engine ranks candidates, while the Verification Engine alone applies ownership rules and controls pickup-code creation.

8 Development Process and Main Risks

8.1 Selected Development Process

The project uses incremental development, as justified for Week 3: requirements and scope are documented early, while implementation is delivered as working increments, without claiming to use the full Scrum or Extreme Programming framework. This fits PrivateFind because the verification rules and hardware integration can be tested and adjusted before the final weeks. Increment 1 (Weeks 7 and 8) ends with the MVP; Increment 2 (Weeks 9 to 11) combines three parallel work streams in the WBS, integration and reuse, security and final feature development, and ends with the feature-complete system in Week 11.

8.2 Main Risks

Risk	Impact	Mitigation
AI extracts an incorrect attribute	Medium	Store confidence and raw input, allow administrator correction, and test with representative item photos.
False claimant studies public information	High	Hide full photos and distinctive details, require the owner report first, limit attempts and use private evidence plus critical checks.
Generic item has weak evidence	Medium	Never auto-approve weak cases; require administrator review or stronger physical proof.
AI service is slow unavailable or costly	Medium	Use a mock during development, limit images and tokens, retry failed jobs and keep the structured scoring pipeline functional.
Hardware delay or assembly problem	Medium	Keep two compartments and test the locker contract with a mock device before physical integration.
Scope growth	Medium	Exclude custom training, chatbot, rewards, public browsing and mobile application from the MVP.
Campus Wi-Fi blocks the ESP32	Low or Medium	Use a local access point for the demonstration if necessary.
Workload peaks or schedule slip	Medium	Update the Gantt chart every week, move non-critical packages by one week, use fallback options and re-estimate after the MVP in Week 8.

9 References

Boehm, B. W. (1981). Software Engineering Economics. Prentice Hall.

Sommerville, I. Software Engineering, 10th edition. Pearson.

Sejong University, Department of Computer Science & Engineering. Software Engineering Project Proposal Guidelines & Submission Requirements, 2026.

Sejong University, Department of Computer Science & Engineering. Software Engineering Project Guide: Project Schedule, Themes and Tools, 2026.